

EC-333 Microprocessor and Interfacing Techniques

Experiment # 09

Stepper Motor Control

1. Objective/s:

- Change notification interrupts to detect push button presses
- Eliminate multiple button operations using a timer interrupt to implement a non-blocking delay
- Lab Task

2. Theory

Stepper motors are electrical mechanical devices used in many robotics applications. In this lab, we will look at how the PIC32 internal timers can be used to implement both cooperative and preemptive scheduling for a real-time system application. We will use time management to transition between states, giving the appearance of continuous rotation of the motor. The position of the stepper motor shaft is a function of four outputs. These outputs must change in a predefined order to cause the motor shaft to move in discrete steps. The outputs to the stepper motor are controlled using a state machine algorithm that generates output patterns and will be used to control position and angular velocity.

Since the PIC32 outputs represent a form of memory, I find it convenient to use state on-entry or on-exit actions to set the processor pins. An on-entry action sets the phase outputs specified for that case whenever a state (case) is entered. An on-exit action sets the phase outputs specified for that case whenever a next state (case) is set. The DIR and MODE inputs define the next state using “if-else” or a sub level of “switch-case” statements.

An alternate implementation uses a table of output codes and an index that becomes the state. It must be remembered that the Basys MX3 platform does not connect consecutive processor pins on a single port to the stepper motor phases. Hence, the state output table must be replaced with a sequence of bit-banging instructions to individually set each phase output pin.

3. Requirements

1. This make of stepper motor is nominally rated for 1600 steps per revolution +/- 7%. Hence, the range can be from 1488 to 1712 steps per revolution. Determine the number of steps for one complete revolution for the particular motor you are using.
2. BTNR controls the direction of rotor rotation (CW or CCW).
3. BTND controls the stepper motor step mode (full or half step). a. The speed or rotation must be the same regardless of stepper mode operation.

- The speed of rotation is set by the hexadecimal value set on the eight slide switches. SW7 is the most significant bit and SW0 is the least significant bit with the switch weighting, as shown

Stepper motor speed control table.

Switch	SW7	SW6	SW5	SW4	SW3	SW2	SW1	SW0
RPM	16	8	4	2	1	0.5	0.25	0.125

- The speed of the motor is to be displayed on the 4-digit 7-segment LED display in RPM with the decimal point to the right of the third digit
- The four digits of the Basys MX3 7-segment display are continually updated with a 1 ms persistence (each digit must be turned on for 1 ms). The four digits will be lit in a round-robin fashion in a foreground operation managed by the Timer 1 ISR.
- Stepper motor outputs are changed in the Timer 1 ISR using the period as determined by the slide switch settings. The period is determined by converting RPM to ms delay between steps.

4. Construction Phase

This lab should be developed in phases to partition the problem into functions or collection of functions that can be tested separately.

Phase 1

- Develop an expression that converts rotational speed in RPM to ms per step.
- Write an application that initializes all switches as well as BTNR and BTND as PIC32 inputs. Configure all stepper motor connections as outputs.
- Develop a PIC32 application program that causes the stepper motor to take a single step in a specified direction (CW or CCW) in a specified mode (half or full step).
- The rotational direction of the stepper is such that when BTNR is depressed, the motor rotates in a clockwise (CW) direction, otherwise it rotates in a counterclockwise (CCW) direction.
- This program will be operated with a breakpoint set in the infinite loop function to determine if the stepper motor phase connection is correct.
- Using a software delay, determine the number of steps needed for one complete revolution.

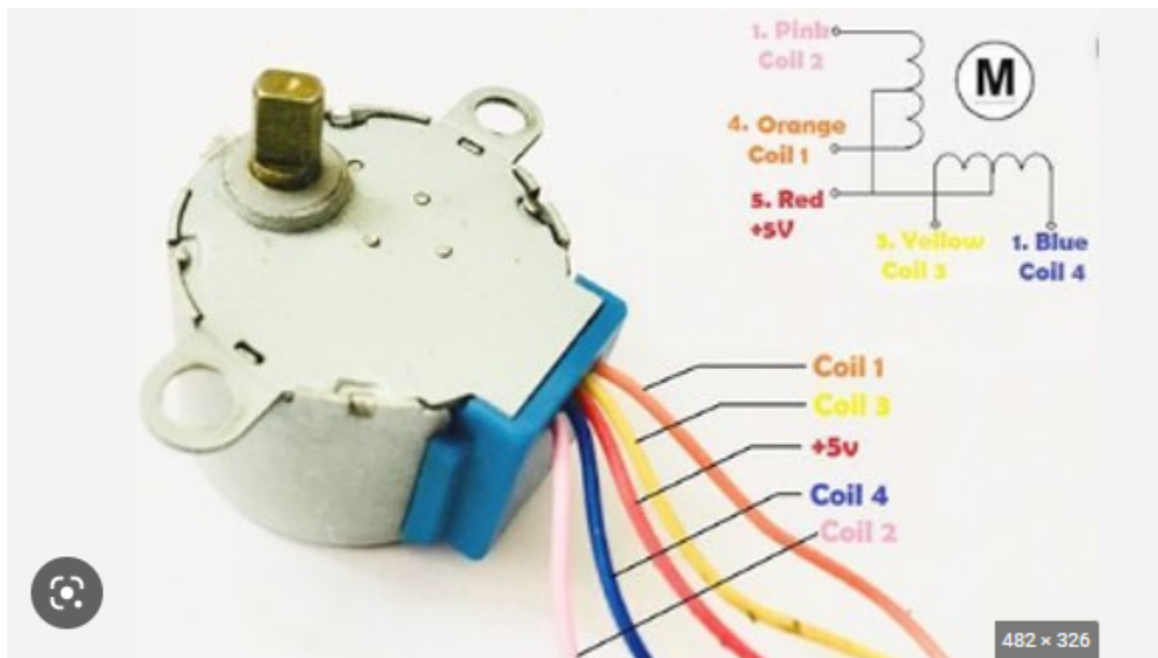
Phase 2

- Initialize Timer 1 to generate a Level 2 interrupt once each 1/10 millisecond.
- Write a Timer 1 interrupt service routine that causes the 7-segment LED to display the RPM (in decimal values) set by the binary encoded switch settings. The LED display is to update one of the four digits once each millisecond.

Phase 3

- Integrate Phase 1 and Phase 2 to implement the following controls in steps two and three.
- The speed of the stepper motor in revolutions per minute (RPM) is set by the binary encoding of the eight slide switch positions. For example, the position of the slide switches results in the stepper motor rotating at 60 RPM. 60 RPM results when SW5, SW4, SW3, and SW2 are set high.

3. The stepper motor normally operates using half steps. If BTND is held depressed, the stepper motor switches to full step operation without altering the RPM set by the slide switches and returns to half step upon release of the button.



5. Lab Task

1. Use the eight slide switches to set the speed of rotation of the stepper motor shaft from 0 to 31.875 RPM in steps of 0.125 RPM. Write a program in which BTND will switch the motor operation from FULL step to HALF step when depressed. BTNR will switch the rotational direction of the rotor from CW to CCW when pressed. The speed of rotation will be displayed on the four-digit seven-segment display digits with 25% duty cycle persistence and one ms update rate.